Tecnológico Nacional de México

Instituto Tecnológico de Chiná

Nombre de la carrera:

Ingeniería en Informática.

Nombre de la asignatura:

Programación cliente - servidor.

Semestre y grupo:

7AI

Actividad:

Investigación de la Unidad 2.

Nombre del alumno:

Aaron Jesús Eloy Aguilar León.

Nombre del docente:

Jenner Noe Che Méndez.

Lugar y fecha:

San Francisco de Campeche, Campeche a 01 de Noviembre de 2026.

Índice

Unidad 2 API HTTP / REST y persistencia de datos..... 3

Subtema 2.1 Recursos, URI, métodos HTTP, encabezados, códigos de estado, JSON, idempotencia y restricciones REST. 3

Subtema 2.2 Diseño API-first con OPENAPI 3.1 o superior compatible: esquemas, parámetros, respuestas, ejemplos y versionado..... 6

Subtema 2.3 Validación de entrada, manejo uniforme de errores, serialización, paginación, filtrado, ordenamiento y búsqueda. 7

Subtema 2.4 Acceso a MySQL/MariaDB o PostgreSQL/Supabase: repositorios, transacciones, migraciones, restricciones e integridad. 8

Subtema 2.5 Separación por capas: rutas/ controladores, casos de uso o servicios, dominio y acceso de datos. 9

Subtema 2.6 Consumo desde HTML/Bootstrap/JavaScript o Flutter/Dart; estados de carga, error, vacío y éxito..... 10

2.7 Pruebas de endpoints y colección reproducible de solicitudes. 11

Referencias..... 12

Unidad 2 API HTTP / REST y persistencia de datos.

Subtema 2.1 Recursos, URI, métodos HTTP, encabezados, códigos de estado, JSON, idempotencia y restricciones REST.

1.- Recursos y URI (Uniform Resource Identifier).

Un recurso es la abstracción fundamental en REST; representa cualquier entidad de información direccionable (ej. estudiante, materia, calificación). La URI es la dirección única con la que el cliente ubica y manipula dicho recurso.

Reglas de diseño de URIs:

- Usar sustantivos en plural, nunca verbos (/estudiantes en lugar de /obtenerEstudiantes).
- Emplear letras minúsculas y separar palabras con guiones medios (-).
- Representar relaciones jerárquicas: /estudiantes/{id}/calificaciones.

2.- Métodos HTTP.

Método.	Operación.	Ejemplo.	Descripción.
GET.	Lectura.	GET /api/v1/estudiantes.	Obtiene uno o varios recursos sin alterar el servidor.
POST.	Creación.	POST /api/v1/estudiantes.	Crea un recurso nuevo dentro de una colección.
PUT.	Reemplazo tota.	PUT /api/v1/docentes/10.	Sustituye completamente la entidad existente.
PATCH.	Edición parcial.	PATCH /api/v1/estudiantes/5.	Actualiza únicamente los campos enviados.

DELETE.	Eliminación.	DELETE /api/v1/materias/MAT-10.	Remueve el recurso especificado del sistema.

3.- Idempotencia y Seguridad.

- Método Seguro: No altera el estado persistente del servidor (GET, HEAD, OPTIONS).
- Método Idempotente: Ejecutar la petición una vez o N veces consecutivas con el mismo cuerpo produce exactamente el mismo efecto final en el servidor (GET, PUT, DELETE). POST no es idempotente.

4.- Encabezados HTTP (Headers).

Metadatos que contextualizan la comunicación:

- Content-Type: application/json: Formato del cuerpo enviado por el cliente o servidor.
- Accept: application/json: Formato que el cliente espera recibir.
- Authorization: Bearer <token_jwt>: Credencial de autenticación del usuario.

5.- Códigos de Estado HTTP.

Se presentan diversos códigos de forma específica, para cada escenario que se presente:

- 2xx (Éxito): 200 OK (Lectura/Actualización exitosa), 201 Created (Creación tras POST), 204 No Content (Eliminación exitosa sin cuerpo de retorno).
- 4xx (Errores del Cliente): 400 Bad Request (Sintaxis o formato inválido), 401 Unauthorized (Autenticación ausente/inválida), 403 Forbidden (Sin permisos), 404 Not Found (Recurso no encontrado), 422 Unprocessable Entity (Error de validación de negocio).
- 5xx (Errores del Servidor): 500 Internal Server Error (Fallo interno sin controlar), 503 Service Unavailable (Servicio saturado o en mantenimiento).

6.- JSON (JavaScript Object Notation).

Formato liviano basado en texto plano para el intercambio de información estructurada mediante pares clave-valor y arreglos.

7.- Las 6 Restricciones Arquitectónicas REST.

1.- Cliente-Servidor: Separación de la interfaz de usuario de la persistencia de datos.

2.- Sin Estado (Stateless): El servidor no guarda sesión del cliente; cada petición lleva toda su información de contexto.

3.- Almacenable en Caché: Las respuestas indican explícitamente si se pueden guardar temporalmente.

4.- Interfaz Uniforme: Identificación clara de recursos, mensajes autodescriptivos y HATEOAS.

5.- Sistema en Capas: La comunicación puede atravesar proxies o gateways de forma transparente.

6.- Código bajo Demanda (Opcional): El servidor envía scripts ejecutables al cliente.

Subtema 2.2 Diseño API-first con OPENAPI 3.1 o superior compatible: esquemas, parámetros, respuestas, ejemplos y versionado.

1.- Metodología API-First.

Consiste en diseñar, documentar y pactar el contrato de la API (mediante especificaciones estandarizadas) antes de escribir código de implementación en el backend o frontend.

2.- Especificación OpenAPI 3.1 (OAS 3.1).

Estándar en YAML o JSON alineado con JSON Schema para estructurar contratos web:

- paths: Define los endpoints disponibles y sus verbos HTTP.
- components/schemas: Modelos de datos reutilizables (entidades y DTOs).
- parameters: Definición de entradas (path, query, header, cookie).
- responses: Mapeo de códigos de estado HTTP esperados y sus respuestas.
- examples: Datos de muestra integrados para documentación interactiva (Swagger UI / Redoc).

3.- Estrategias de Versionado de APIs.

- Por Path (Recomendado): /api/v1/estudiantes.
- Por Encabezado Personalizado: X-API-Version: 1.0.
- Por Negociación de Contenido (Accept): Accept: application/vnd.sia. v1+json.

Subtema 2.3 Validación de entrada, manejo uniforme de errores, serialización, paginación, filtrado, ordenamiento y búsqueda.

1.- Validación de entrada.

Proceso de verificación en la capa de control para asegurar que los datos recibidos en el body o query params cumplan con tipos, rangos y restricciones requeridas antes de llamar a la capa de negocio.

2.- Manejo Uniforme de Errores (RFC 7807 - Problem Details).

Estándar estructurado para responder ante fallos:

```JSON
{
  "type": "https://api.sia.itchina.edu.mx/errors/validation-error",
  "title": "Error de validación en la solicitud",
  "status": 400,
  "detail": "El campo 'promedio' debe ser un número entre 0 y 100.",
  "instance": "/api/v1/calificaciones"
}
```

3.- Serialización y Deserialización.

Transformación bidireccional entre instancias de objetos de código (Python, Node.js) y cadenas JSON, utilizando DTOs (Data Transfer Objects) para ocultar datos sensibles como credenciales.

4.- Operaciones avanzadas en colecciones.

- Paginación: Basada en desplazamiento (/estudiantes?page=2&limit=20) o en cursores (/estudiantes?cursor=eyJpZCI6MTB9).
- Filtrado: Restricción de resultados por campos (/estudiantes?carrera=sistemas&estatus=activo).
- Ordenamiento: Dirección y atributo (/estudiantes?sort=-promedio,nombre).
- Búsqueda: Búsqueda abierta de texto (/estudiantes?q=Aguilar).

Subtema 2.4 Acceso a MySQL/MariaDB o PostgreSQL/Supabase: repositorios, transacciones, migraciones, restricciones e integridad.

1.- Patrón Repositorio (Repository Pattern).

Capa de abstracción que encapsula el acceso a la base de datos (mediante SQL directo u ORMs), aislando la lógica de negocio del gestor de persistencia utilizado.

2.- Transacciones ACID.

Conjunto de operaciones ejecutadas como una sola unidad atómica (BEGIN, COMMIT, ROLLBACK).

Es por esto, que se garantiza la consistencia en operaciones complejas como inscripciones o transferencias.

3.- Migraciones de Base de Datos.

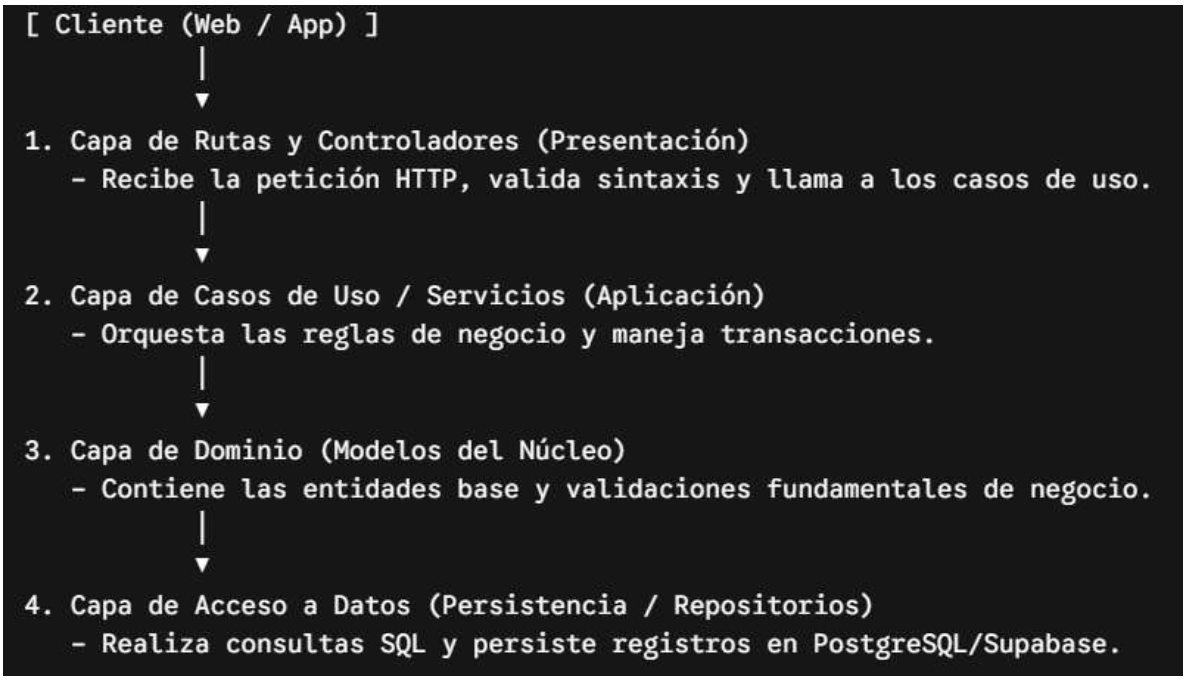
Archivos de control de versiones del esquema DDL ejecutados en orden cronológico para garantizar sincronización entre los entornos de desarrollo, pruebas y producción.

4.- Restricciones e Integridad Referencial.

- Claves Primarias (PK), Claves Foráneas (FK) con reglas de borrado (CASCADE, RESTRICT).
- Restricciones NOT NULL, UNIQUE y CHECK (ej. CHECK (calificacion >= 0 AND calificacion <= 100)).
- Row Level Security (RLS) en Supabase/PostgreSQL: Políticas integradas a la base de datos que filtran el acceso a filas según el ID del usuario (auth.uid ()) o su rol.

Subtema 2.5 Separación por capas: rutas/ controladores, casos de uso o servicios, dominio y acceso de datos.

Estructuración bajo el principio de Clean Architecture / Layered Architecture:



Subtema 2.6 Consumo desde HTML/Bootstrap/JavaScript o Flutter/Dart; estados de carga, error, vacío y éxito.

Al conectar un cliente web (Next.js/React) o móvil (Flutter/Dart) con la API REST, la interfaz debe gestionar en su estado reactivo cuatro escenarios obligatorios:

- Estado de Carga (Loading State): Visualización de spinners o skeletons mientras la solicitud HTTP está en vuelo.
- Estado de Éxito (Success State): Renderizado de los datos recibidos ante respuestas con códigos 2xx.
- Estado de Error (Error State): Despliegue de alertas o mensajes amigables al capturar códigos de error (4xx/5xx) o fallos de red.
- Estado Vacío (Empty State): Interfaz diseñada para notificar al usuario cuando la respuesta es exitosa (200 OK) pero la lista contiene 0 elementos.

2.7 Pruebas de endpoints y colección reproducible de solicitudes.

1.- Herramientas de Pruebas de API.

Uso de clientes como Postman, Insomnia o Bruno para la ejecución y validación de peticiones HTTP en entornos aislados.

2.- Estructuración de Colecciones Reproducibles.

Organización: Agrupación lógica de carpetas por módulos (Auth, Estudiantes, Calificaciones).

Variables de Entorno (Environments): Parametrización dinámica de URLs base ({{base_url}}) y tokens de sesión ({{bearer_token}}) para alternar rápidamente entre entornos local, pruebas y producción.

Pruebas Automatizadas (Assertions): Scripts adjuntos a la colección que verifican automáticamente el código de estado devuelto, el tiempo de respuesta y el esquema del JSON retornado.

Referencias.

1.- Martin, R. C. (2019). Arquitectura limpia: Guía para el desarrollo y diseño de software. B&B Books.

2.- Pressman, R. S., & Maxim, B. R. (2010). Ingeniería del software: Un enfoque práctico (7.ª ed.). McGraw-Hill Interamericana.

3.- Piattini Velthuis, M., & García Rodríguez, F. (2005). Calidad y modelos de evolución del software. Editorial Ra-Ma.